

Lab 04: Configure ALB with Google Cloud Armor

Build a Secure Google Cloud Network | GSP215

Overview

Configured a Global External Application Load Balancer with backends in two regions. Stress tested the load balancer using siege to trigger cross-region failover, then used Cloud Armor to denylist the stress test VM's IP address at the edge.

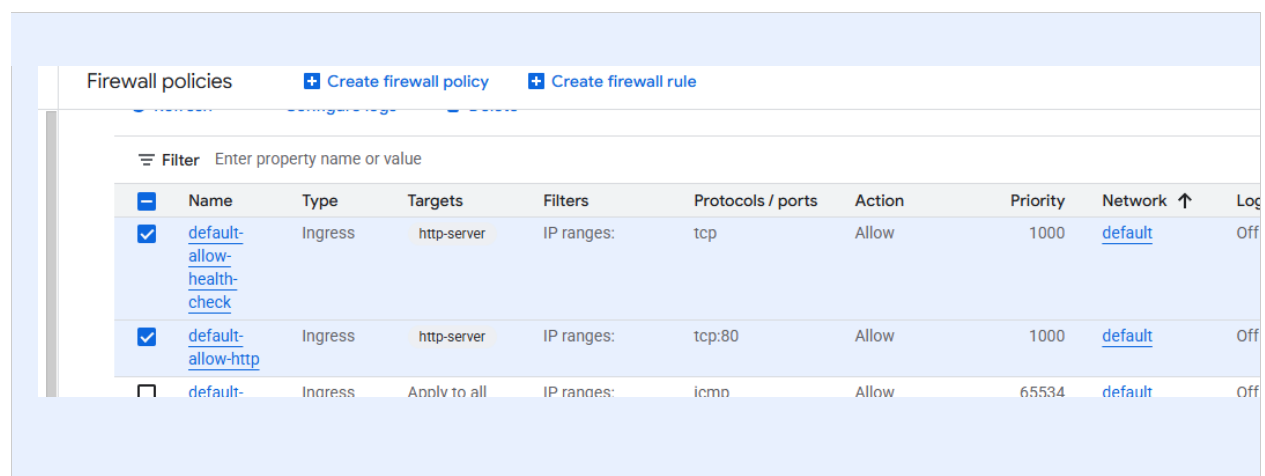
Objectives

- Configure HTTP and health check firewall rules
- Create instance templates and managed instance groups in 2 regions
- Configure a Global External Application Load Balancer
- Stress test the load balancer to trigger regional failover
- Create a Cloud Armor security policy to denylist an IP
- Verify traffic is blocked and explore security logs

Task 1 - Configure Firewall Rules

Created two firewall rules to allow HTTP traffic and health checks to reach backend instances.

- default-allow-http: TCP port 80 from 0.0.0.0/0 targeting http-server tag
- default-allow-health-check: TCP from 130.211.0.0/22 and 35.191.0.0/16 (Google health check ranges)

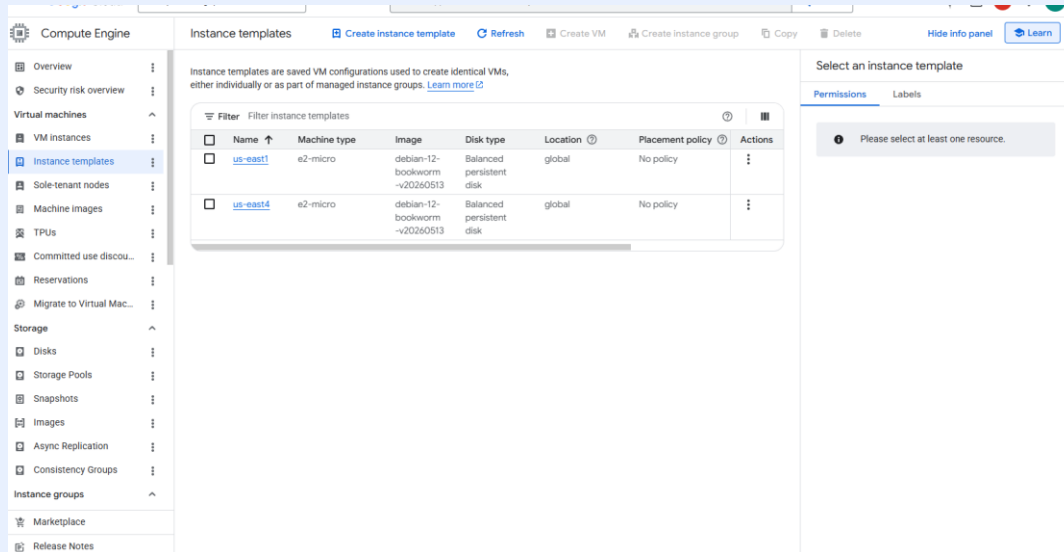


Firewall policies + Create firewall policy + Create firewall rule									
Filter Enter property name or value									
☐	Name	Type	Targets	Filters	Protocols / ports	Action	Priority	Network	Log
☒	default-allow-health-check	Ingress	http-server	IP ranges:	tcp	Allow	1000	default	Off
☒	default-allow-http	Ingress	http-server	IP ranges:	tcp:80	Allow	1000	default	Off
☐	default	Ingress	Apply to all	IP ranges:	icmp	Allow	65534	default	Off

Task 2 - Configure Instance Templates and Groups

Created instance templates for two regions and deployed managed instance groups with autoscaling.

- Region 1 template: e2-micro, http-server tag, Apache startup script from GCS
- Region 2 template: same config with Region 2 subnet
- Both MIGs: min 1, max 2 instances, autoscale on 80% CPU



 [Screenshot: Apache page showing Client IP, Hostname, and Server Location]

Replace this with your screenshot

Task 3 - Configure the Application Load Balancer

Created a Global External Application Load Balancer named http-lb with backends in both regions.

- Frontend: HTTP, IPv4 and IPv6, port 80
- Region 1 backend: Rate balancing mode, max 50 RPS
- Region 2 backend: Utilization mode, max 80% CPU
- Health check: TCP port 80
- Logging: Enabled with 100% sample rate

Load balancing
+ Create load balancer
Refresh
Delete



Monitor + Analyze Load Balanced Traffic

Use VPC Flow Logs and Flow Analyzer to gain flow level insights for your Load Balancers and backends. [Learn more](#)

- ✓ Diagnose connectivity issues
- ✓ Identify top talkers + Analyze network utilization
- ✓ For passthrough load balancers, understand how flows are being distributed among their backends

[Try now](#)
[Remind me later](#)

Load balancers
Backends
Frontends
Service LB policies

Filter Enter property name or value

☐	Name	Load balancer type	Access type	Protocols	Region	Backends	Actions
☐	http-lb	Application	External	HTTP		✓ 1 backend service (2 instance groups, 0 network endpoint groups)	⋮

To view or delete load balancing resources like forwarding rules and target proxies, go to the [load balancing components view](#).

Task 4 - Test the Load Balancer

Accessed the load balancer via browser and ran a siege stress test from a VM in Region 3 to trigger cross-region failover.

```
sudo apt-get update && sudo apt-get -y install siege
export LB_IP=[LB_IP_v4]
siege -c 150 -t120s http://$LB_IP
```

Results: Initially all traffic went to Region 1 backend. As RPS exceeded 50, traffic spilled over to Region 2 backend.

The screenshot displays the Google Cloud Logs Explorer interface. At the top, there's a search bar and filters for 'Project logs'. Below this, a query is entered: `resource.type="http_load_balancer"`, `resource.labels.forwarding_rule_name="http-lb"`, and `resource.labels.url_map_name="http-lb"`. The interface shows a timeline view with a log entry at 2026-05-27 23:27:35.494. The log entry details show an HTTP GET request to `http://8.233.194.201/` with a 200 status and a response body containing JSON data.

Task 5 - Denylist the Siege VM with Cloud Armor

Created a Cloud Armor security policy to block the siege VM's IP at the Google edge before it reaches the backends.

- Policy name: denylist-siege
- Default rule: Allow
- Custom rule: Deny 403 for siege VM IP at priority 1000
- Target: http-backend

```
curl http://$LB_IP
```

Result: siege VM received 403 Forbidden. Browser access still worked via default allow rule.

[illegible]

Real World Application

- E-commerce sites use Cloud Armor to block DDoS attacks and scrapers at the edge
- Cross-region load balancing ensures availability even if one region goes down
- Rate-based backend policies prevent any single backend from being overwhelmed
- Security policy logs provide an audit trail of blocked traffic for compliance

What I Learned

- Global ALBs terminate traffic at the Google Edge and proxy it over Google's private network
- Cross-region failover triggers automatically when a backend exceeds its capacity threshold
- Cloud Armor blocks traffic at Layer 7 before it reaches the VPC or backends
- Denylisting at the edge saves backend resources from processing malicious requests
- Security policy logs show exactly which requests were denied and why
- Rate mode and utilization mode are two different backend balancing strategies

Resources

GCP Cloud Armor Docs: <https://cloud.google.com/armor/docs>

GCP Application Load Balancer Docs: <https://cloud.google.com/load-balancing/docs>

Lab ID: GSP215